

INDUSTRIAL INTERNET OF THINGS

POSITION MONITORING SYSTEM

PROJECT REPORT

Submitted by:

E . Lakshmi Triveni

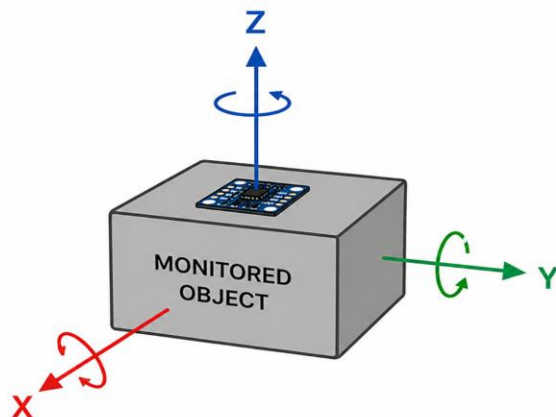
Department of - Electronics and communication

Engineering

VIT-AP university

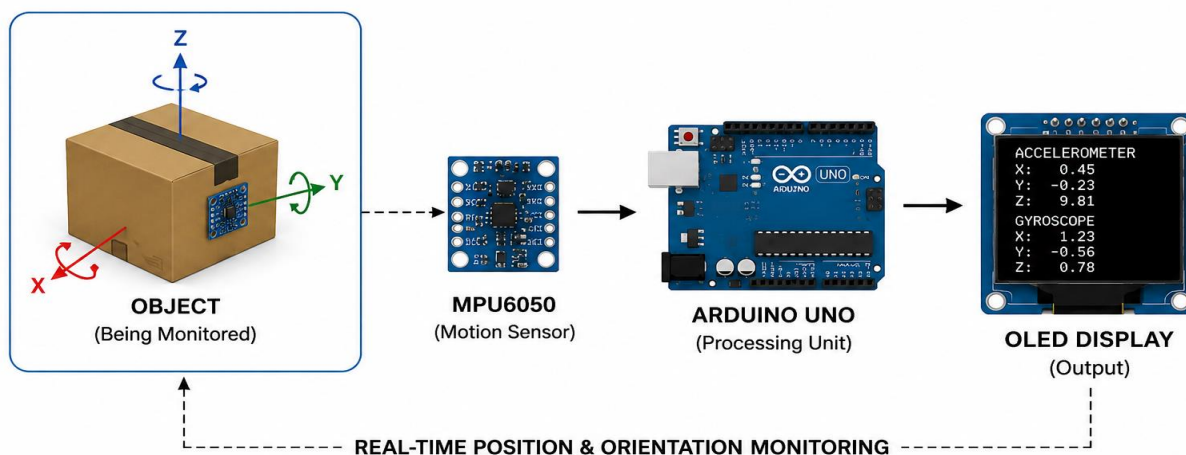
PROBLEM STATEMENT:

- Industries require continuous monitoring of object position and orientation to ensure operational efficiency, safety, and reliability.
- Traditional monitoring systems are often expensive and complex, creating a need for a simple and cost-effective real-time monitoring solution.
- Therefore, this project develops a cost-effective solution to monitor real-time position and orientation data



SCOPE OF THE SOLUTION:

- Designing and developing a prototype for real-time monitoring of an object's position and orientation.
- Employing an MPU6050 accelerometer and gyroscope sensor with Arduino Uno to acquire and process motion data and display it on an OLED screen.
- Applicable to robotics, industrial automation, machinery monitoring, drones, transportation systems, and educational IoT applications.

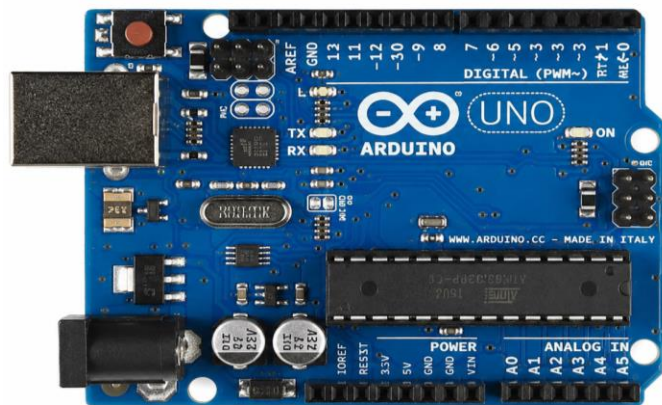


REQUIRED COMPONENTS TO DEVELOP SOLUTION:

- Hardware:

ARDUINO UNO:

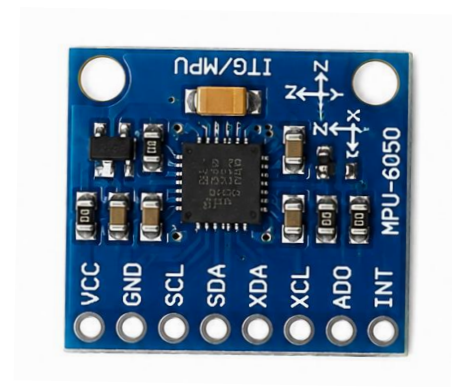
The Arduino Uno serves as the main processing unit of the system. It reads the accelerometer and gyroscope data from the MPU6050 sensor and controls the OLED display to show real-time readings.



MPU6050 SENSOR MODULE:

The MPU6050 is a six-axis motion sensor that combines a three-axis accelerometer and a three-axis gyroscope.

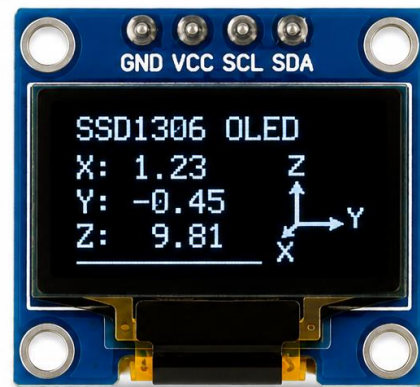
It measures the object's



acceleration and rotational movement along x, y, z axes.

OLED Display (SSD1306):

The OLED display acts as the output interface of the system. It displays the real-time accelerometer and gyroscope values received from the Arduino Uno



JUMPER WIRES:

Jumper wires are used to establish electrical connections between the Arduino Uno, MPU6050 sensor, and OLED display. They enable communication and power transfer among the components.

USB POWER SUPPLY (Virtual in Simulation):

The USB supply provides power to the Arduino Uno and the connected devices. In the Wokwi simulation environment, power is provided virtually.

Software:

WOKWI SIMULATION PLATFORM:

Wokwi is an online electronics simulation platform used to design, simulate, and test the complete circuit without requiring physical hardware. It allows real-time verification of circuit functionality and sensor outputs.

PROGRAMMING LANGUAGE:

Embedded C/C++ is used to write the program that reads sensor data from the MPU6050 and displays the values on the OLED screen.



• **EMBEDDED C** •

LIBRARIES USED:

- Wire.h:

The Wire library implements the I2C communication protocol.

- Adafruit GFX.h:

The Adafruit GFX library provides graphical functions such as displaying text and basic graphics on the OLED screen.

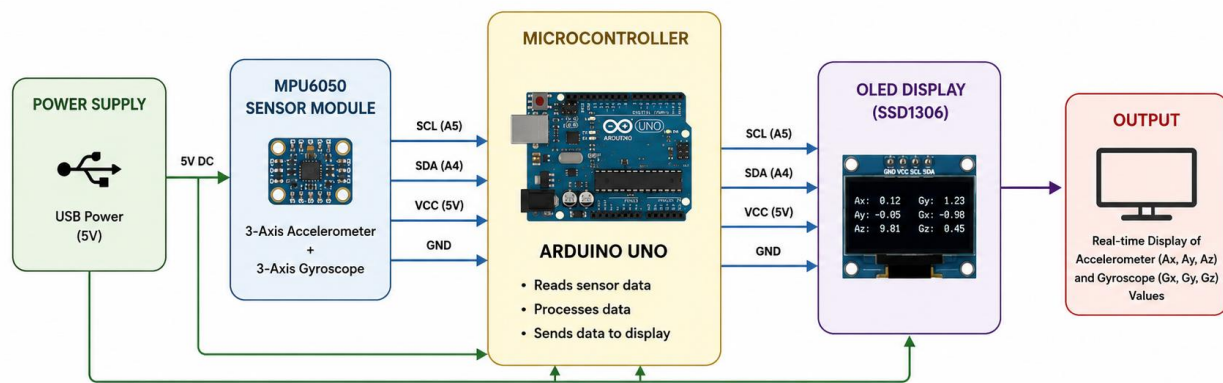
- Adafruit SSD1306.h:

This library controls the SSD1306 OLED display and allows sensor readings to be displayed in a structured format.

- MPU6050.h:

The MPU6050 library provides functions to initialize the sensor and read accelerometer and gyroscope data from the MPU6050 module.

BLOCK DIAGRAM :

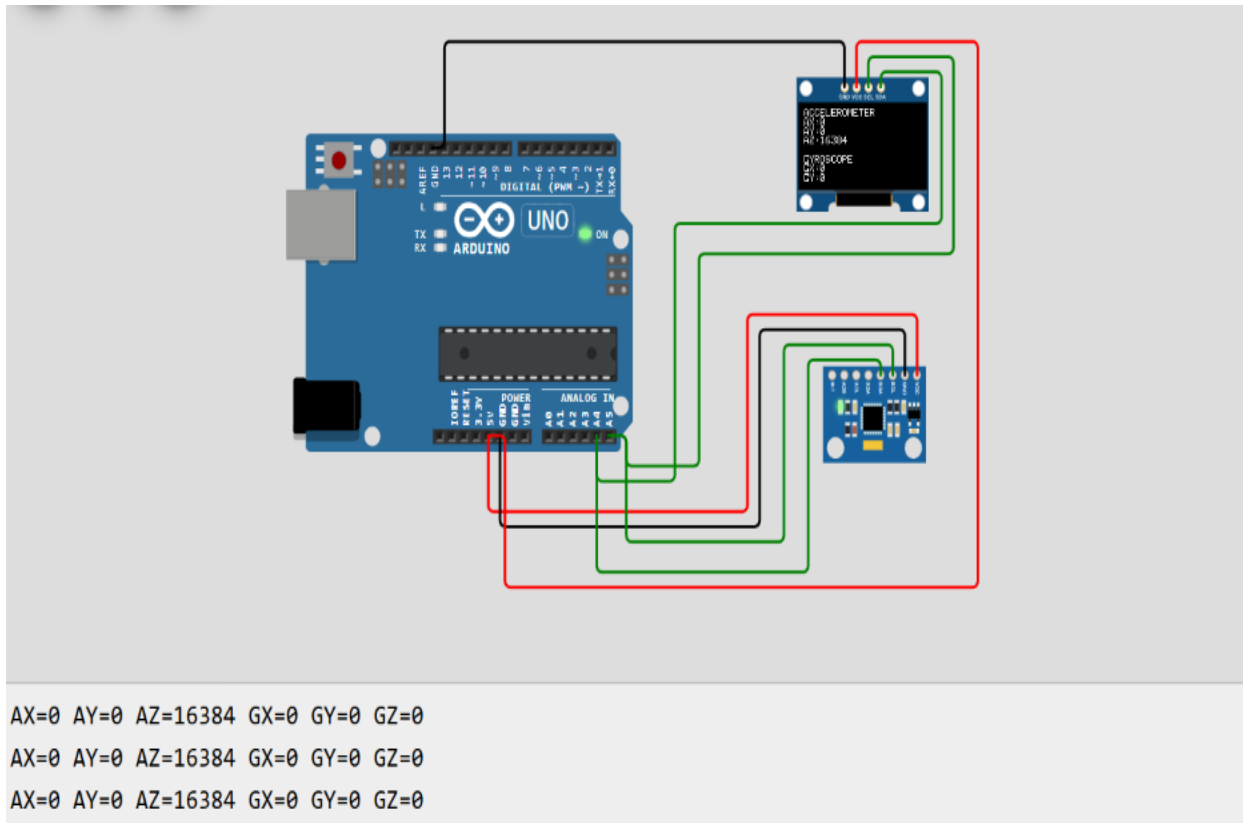


Working principle:

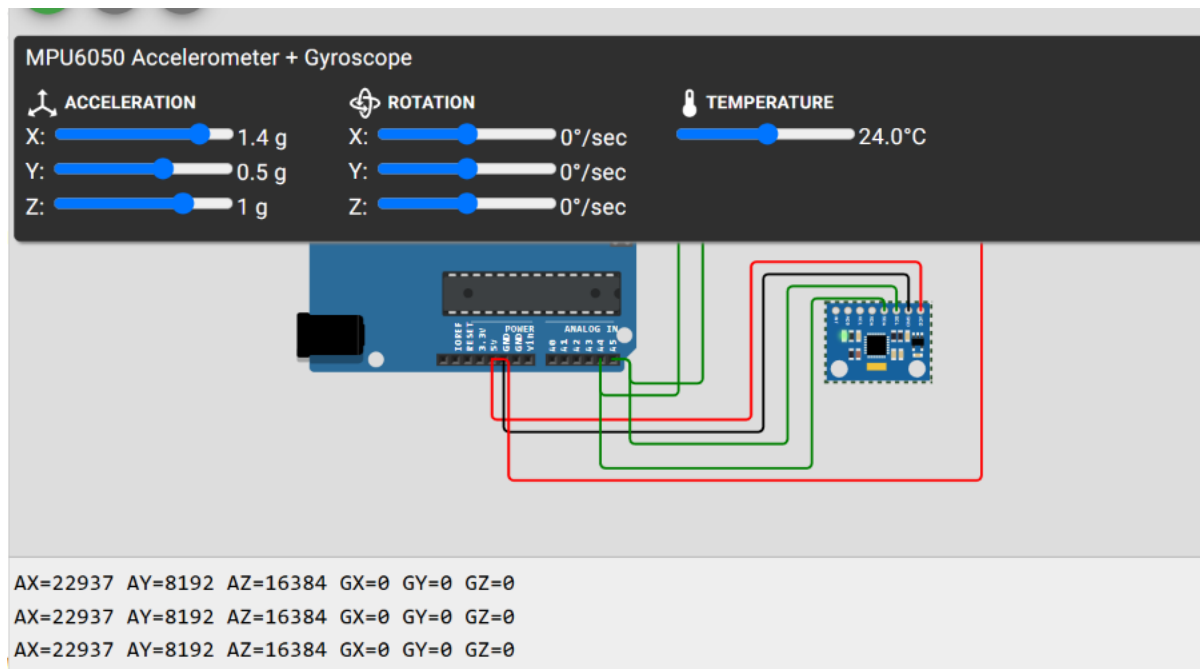
- MPU6050 measures acceleration and rotational data
- Arduino UNO reads processes the data
- Processed data is sent to OLED display
- Real-time values are shown on the OLED screen

SIMULATED CIRCUIT:

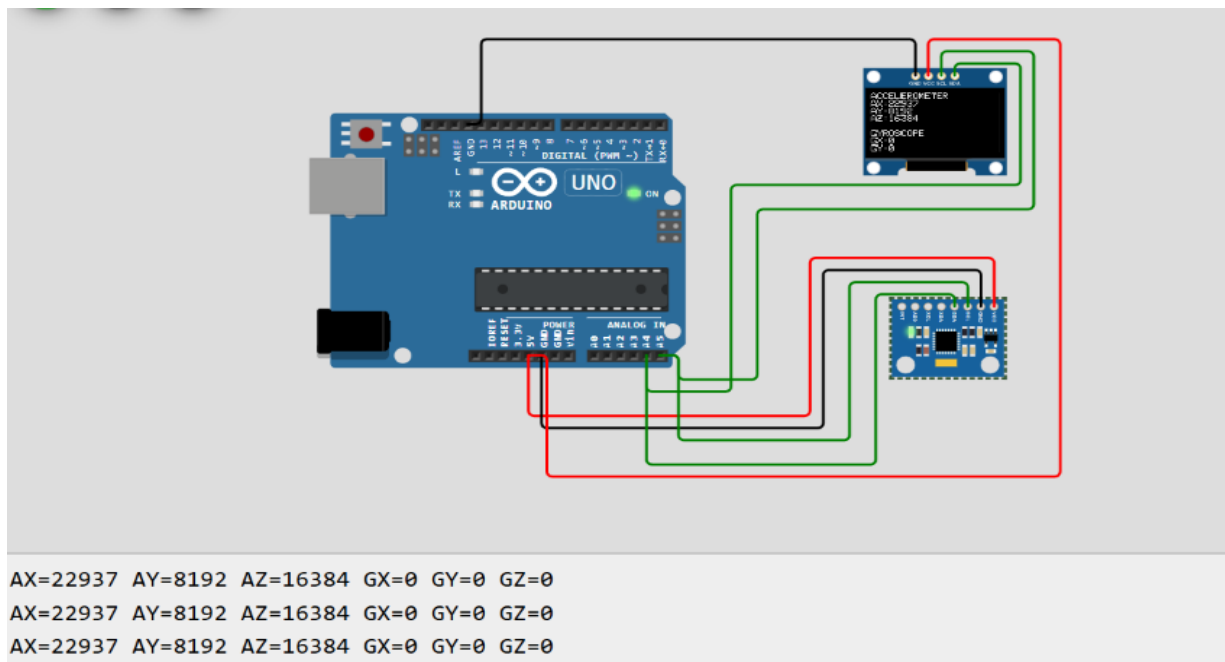
- Initial circuit:



- Simulated circuit and its output after rotating and changing the acceleration values:

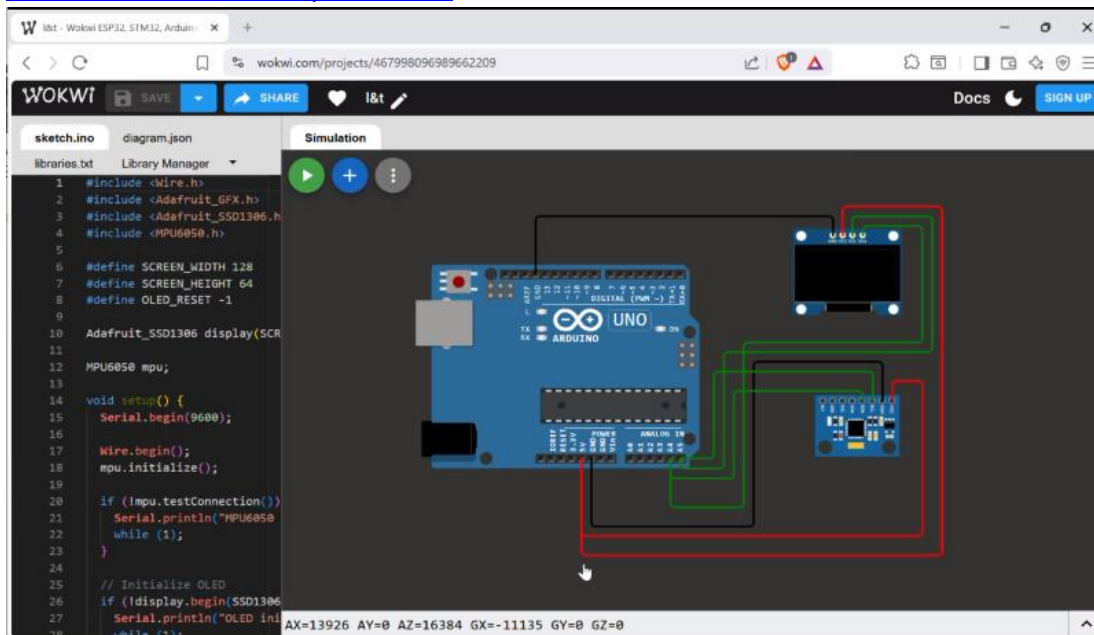


- Its output:



VIDEO OF THE DEMO:

[click here to watch complete video](#)



- Since the project is simulation-based, the circuit design, program development, compilation, and testing were performed entirely using the Wokwi simulation platform, and no physical hardware implementation was required.

CODE OF THE SOLUTION:

```
// Include the required libraries
```

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <MPU6050.h>
```

```
// Define OLED parameters
```

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
```

```
// Create OLED and MPU6050 objects
```

```
Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  OLED_RESET
);
MPU6050 mpu;
```

```
// Initialize Serial communication,
```

// I2C communication and MPU6050

void setup()

{

Serial.begin(9600);

Wire.begin();

mpu.initialize();

}

// Check sensor and display connection

if (!mpu.testConnection())

{

Serial.println(

"MPU6050 connection failed!");

while (1);

}

if (!display.begin(

SSD1306_SWITCHCAPVCC,

0x3C))

{

Serial.println(

"OLED initialization failed!");

while (1);

}

```
// Display project title
display.clearDisplay();
display.setTextColor(
SSD1306_WHITE);
display.setCursor(0,0);
display.println(
"IIoT Position");
display.println(
"Monitoring");
display.display();
delay(2000);
```

```
// Declare sensor variables
int16_t ax, ay, az;
int16_t gx, gy, gz;
```

```
// Read sensor values
mpu.getMotion6(
&ax, &ay, &az,
&gx, &gy, &gz
);
```

```
Nxt section is to display the sensor data on OLED
display.clearDisplay();
display.setTextSize(1);
```

```
display.setCursor(0,0);  
display.println(  
"ACCELEROMETER");  
display.print("AX:");  
display.println(ax);  
display.print("AY:");  
display.println(ay);  
display.print("AZ:");  
display.println(az);  
display.println();  
display.println("GYROSCOPE")  
display.print("GX:");  
display.println(gx);  
display.print("GY:");  
display.println(gy);  
display.print("GZ:");  
display.println(gz);  
display.display();
```

Similarly next section is to print data on serial monitor

